

Session 06 — Project Guide

TaskFlow: State + Events + Re-render

This guide documents the official **Session 06 End** state of TaskFlow. Everything in it comes from the real project files — nothing is invented, and it does not cover homework. Keep this guide — you can read it at home, days later, and rebuild the whole mental model from it alone.

1. Quick Recap — What Session 05 Already Gave You

Before today, you could already:

- write a normal function component (`Header`),
- describe the exact data a component needs with a typed props `interface` (`StatCardProps` , `TaskItemProps`),
- pass typed data from a parent to a child (`<StatCard label="Total Tasks" value={3} />`),
- read a prop as `props.x` ,
- connect files with `import / export default` .

None of that changes today. Every component you already know how to build is still built exactly the same way. Today adds one new ability on top of everything above: **a component that can remember something, and change it.**

2. Why Static React UI Isn't Enough

Session 05 ended with a real, working page that looked complete — but clicking a filter button or typing in the search box did nothing. That wasn't a bug. It was a real limitation of everything you knew at the time.

Here's exactly why. A component is a function. Every time it runs, it reads whatever values exist at that moment and returns TSX built from them. But a plain variable declared inside a function disappears the moment the function finishes running — and worse, even if it didn't disappear, *changing* a plain variable never tells React "hey, go run this function again so the screen updates." Nothing was watching for the change. Nothing could react to it.

That missing piece — a way for a component to hold onto a value **and** have React notice when it changes — is called **state**, and `useState` is the tool that gives you both halves of it at once.

3. The Old Model vs. the New Model

You've actually seen this exact shape before, in Session 04C — just done by hand:

SESSION 04C (plain TypeScript):

```
currentFilter = "completed";  
renderTasks();
```

ACTION → CHANGE DATA/VARIABLE → YOU MANUALLY CALL RENDER → UI CHANGES

SESSION 06 (React):

```
setCurrentFilter("completed");
```

ACTION → CALL A STATE SETTER → REACT RENDERS AGAIN, AUTOMATICALLY → UI CHANGES

The overall shape — something happens, data changes, the screen updates — has not changed at all. **The only thing that's different is who is responsible for the "render again" step.** In Session 04C, you called `renderTasks()` yourself, every single time, and forgetting it was a real, common bug. Starting today, calling the state's own setter function is the *only* thing you have to remember — React takes care of re-running your component and updating the screen by itself.

4. `useState` — Current Value, Setter, Initial Value

Here's the real code from `App.tsx`:

```
const [currentFilter, setCurrentFilter] = useState("all");
```

This one line does three things at once, and it's worth naming each part separately, because this exact shape repeats every single time you add a new piece of state:

- `useState("all")` — creates one piece of state, starting with the value `"all"`. That starting value is called the **initial value**, and it is only ever used the very first time this component renders.
- `currentFilter` — the **current value**. Whatever this state holds right now, read the same way you'd read any other variable.

- `setCurrentFilter` — the **setter**. The one and only function allowed to actually change this piece of state.

`useState` always hands back exactly these two things, in exactly this order, inside an array — which is why the syntax on the left looks like `const [value, setValue] = ...`. You'll see this same shape for every piece of state in this project: `searchText / setSearchText`, `showTasks / setShowTasks`.

Why does calling the setter matter? You cannot replace the state value by writing `currentFilter = "completed";` — `currentFilter` is a `const` binding, so TypeScript rejects that assignment; the line does not even compile. React state is changed through the setter instead:

```
setCurrentFilter("completed");
```

Calling the setter also tells React that this piece of state changed and schedules an update. The setter is not just "a way to change the value" — it is the specific signal that tells React "this component's data changed, please run it again."

5. The Re-render Mental Model

Once a setter is called, here's what actually happens, in order:

1. `setCurrentFilter("completed")` runs.
2. React updates `currentFilter`'s stored value to `"completed"`.
3. React runs your `App` function again, from the top, exactly like it did the very first time — except now, every line that reads `currentFilter` sees the new value.
4. React takes the new TSX description that running `App` again produced, and updates the parts of the browser DOM that need to change to match it.

You never write the "run it again" step yourself. It's automatic, and it's the entire reason `useState` exists instead of a plain variable. The pattern to remember: **state setter** → **component runs again** → **React sees the new UI description** → **React updates what needs to change**.

6. Normal Named Event Handlers

Every handler this session is a normal, named function — nothing new syntactically, just a function you already know how to write, given a job to do when something happens:

```
function handleShowAll() {
  setCurrentFilter("all");
}
```

```
<button
  className={"filter-button" + (currentFilter === "all" ? " active" : "")}
  onClick={handleShowAll}
>
  All
</button>
```

`onClick={handleShowAll}` hands React **the function itself** — not the result of calling it — so React can call it later, exactly once, at the moment the button is actually clicked.

This is the single most important syntax detail this session. Compare:

```
onClick={handleShowAll}      // correct – the function itself
onClick={handleShowAll()}    // wrong – calls it immediately, while rendering
```

`onClick={handleShowAll()}` calls the function immediately while React is rendering the component, instead of giving React a function to call later when the user clicks. Because this particular function changes state (it calls `setCurrentFilter`), calling it during rendering can also cause React to render repeatedly and produce an error — the call changes state, React renders again to reflect that, this same line runs again, and the cycle repeats. The correct pattern is always the same: pass the function itself, with no parentheses, and let React decide when to call it — `onClick={handleShowAll}`.

7. Filter-Button State and Conditional Styling

App has three nearly-identical handlers, one per filter:

```
function handleShowAll() {
  setCurrentFilter("all");
}

function handleShowCompleted() {
  setCurrentFilter("completed");
}
```

```
function handleShowPending() {
  setCurrentFilter("pending");
}
```

Each button reads `currentFilter` back to decide its own CSS class:

```
className={"filter-button" + (currentFilter === "all" ? " active" : "")}
```

Read this as plain string-building: start with `"filter-button"`, then — only if `currentFilter` currently equals `"all"` — add the extra text `" active"` onto the end. The button element itself never disappears and never changes *which* element it is; only the string assigned to `className` changes, which is why this is called **conditional styling**, not conditional rendering (Section 11 draws that line precisely).

8. `ChangeEvent<HTMLInputElement>` and `event.target.value`

The search box needs to know *what was typed*, not just *that* something was typed — which means its handler needs to receive the actual browser event:

```
function handleSearchChange(event: ChangeEvent<HTMLInputElement>) {
  setSearchText(event.target.value);
}
```

This is genuinely new syntax this session — nothing in Session 05 used it. Explained at exactly the level you need:

- `import type { ChangeEvent } from "react";` — imports TypeScript *type information* only; `import type` never brings in anything that exists while the app is actually running, only a shape TypeScript checks against.
 - `ChangeEvent` — React's type for "something changed on a form element."
 - `<HTMLInputElement>` — narrows that down to "specifically, this event came from an `<input>` element" — which matters because `.value` (the next line) only exists on input-like elements, not on every possible element a change event could theoretically come from.
 - `event.target.value` — `event.target` is the actual `<input>` element the user just typed into; `.value` is the text currently sitting inside it, read fresh, at the exact moment this handler runs.
-

9. Controlled Inputs — `value` + `onChange` Together

```
<input
  type="text"
  className="search-input"
  placeholder="Search tasks..."
  value={searchText}
  onChange={handleSearchChange}
/>
```

This pairing has a name: a **controlled input**. `value={searchText}` means React state is the *single source of truth* for what the box displays — not whatever the browser feels like showing. `onChange={handleSearchChange}` is what keeps that state accurate: every keystroke fires it, and it immediately calls `setSearchText(event.target.value)`, updating state to match what was just typed.

Trace the whole loop out loud, because this exact loop repeats for every controlled input you'll ever write:

```
you type a character
  → onChange fires
  → handleSearchChange runs
  → setSearchText(event.target.value) is called
  → React re-renders App
  → the input's value={searchText} reads the new state
  → the box shows what you typed
```

What happens if `value=` is removed but `onChange` stays? The input becomes **uncontrolled**. Typing may still appear to work — the browser's own default input behavior takes over showing what you type — and `onChange` may still fire, updating `searchText` in React state. But React is no longer deciding what value the box displays; the browser is. The state → displayed-input connection is broken, even though the state itself is still being updated correctly. This difference becomes especially important when code later changes or resets the state — the input would no longer show that new value, because it isn't reading from state anymore.

What happens if `onChange=` is removed but `value=` stays? The opposite problem: the input becomes frozen. React insists the box always show exactly `searchText`, and since nothing ever calls `setSearchText` anymore, that value can never change — typing does nothing at all. (React will also warn you about this exact situation in the browser console.)

Both halves matter, and they do two different jobs: `value` controls what's *displayed*; `onChange` is what keeps the underlying *state* accurate.

10. Conditional Rendering — Ternary to `null`

Below the search box, a feedback message appears only while there's something typed:

```
{searchText !== "" ? (  
  <p className="search-feedback">Searching for: {searchText}</p>  
) : null}
```

Read the `{}` here exactly like Session 05's `{}` — "evaluate a real TypeScript value here" — except this time the value is a ternary expression: "if `searchText` is not an empty string, produce this `<p>` element; otherwise, produce `null`." `null` is React's way of saying "render nothing at all here" — not an empty string, not a hidden element, but genuinely no element in the page at all.

11. Conditional Styling vs. Conditional Rendering

These two ideas both use a ternary, and it's easy to blur them together — so it's worth stating the difference plainly, since Session 06 is the session that introduces the second one for real:

	Conditional styling (Section 7)	Conditional rendering (Section 10)
What changes	A string (the CSS class)	Whether the element exists at all
Element itself	Always present on the page	Present or completely absent
Example	<code>className={"filter-button" + (... ? " active" : "")}</code>	<code>{searchText !== "" ? <p>...</p> : null}</code>

A filter button never disappears — it just visually looks "active" or not. The search-feedback paragraph genuinely does not exist in the page at all until you type something.

12. Teamwork — Boolean State and the Toggle Pattern

Teamwork extended the exact same recipe to a new, self-contained feature: hiding and showing the whole task list.

```
const [showTasks, setShowTasks] = useState(true);
```

Same shape as `currentFilter / searchText` — current value, setter, initial value — just holding a `boolean` (`true / false`) instead of a `string` this time. `useState` works with any kind of value; nothing about the pattern itself changes.

```
function handleToggleTasks() {  
    setShowTasks(!showTasks);  
}
```

`!showTasks` reads as "*the opposite of whatever `showTasks` currently is*" — the exact same `!` operator already familiar from Session 04C's `if (!x)` guards, now flipping a boolean instead of checking one. This one line correctly handles both directions (hiding and showing again) without ever needing to know which direction it currently is in.

```
<button className="toggle-tasks-button" onClick={handleToggleTasks}>  
    {showTasks ? "Hide tasks" : "Show tasks"}  
</button>  
  
{showTasks ? (  
    <ul className="task-list">  
        { /* the three task rows */ }  
    </ul>  
    ) : null}
```

The button's own **text** is a ternary choosing a string (conditional styling's cousin — choosing text instead of a class). The task list itself is wrapped in the exact same ternary-to-`null` shape from Section 10 — the whole `<ul>`, and everything inside it, exists or doesn't exist depending on `showTasks`.

13. Temporary Teaching Features — Named Honestly

Two things you built today are **not** part of TaskFlow's real, final destination app — they exist purely to give you something concrete to practice on:

- **"Searching for: ..." feedback** — exists only to demonstrate real conditional rendering with an obvious, visible result.

- **Show/Hide Tasks** — exists only to give teamwork a practice target using the exact pattern just built live.

Neither is a mistake, and neither should be treated as "extra" or apologized for — they were built specifically to teach these two concepts clearly. A later session will eventually replace both with TaskFlow's real, permanent behavior, through real, visible, taught changes — never silently.

14. Exact Session 06 End Behavior

- All three filter buttons visually highlight correctly when clicked.
- The search box holds whatever you type, and a "Searching for: ..." message appears below it while there's text, disappearing the instant the box is empty.
- The Show/Hide Tasks button hides and shows the whole task list, and its own label changes to match ("Hide tasks" / "Show tasks").
- **The three task rows are still completely hardcoded.** Clicking a filter or typing in the search box changes the *button's highlight* and the *feedback message* — it does **not** change which task rows are visible. Stats stay fixed at 3 / 1 / 2, exactly as in Session 05.

Why don't the filters and search actually filter the task list yet? Because doing that means taking a real collection of tasks and deriving a *subset* of it based on state — and that requires `.map()` to render a variable number of rows, which hasn't been taught yet. Today's filter/search state genuinely works — it updates, and you can see it working via the highlight and the feedback message — it just isn't yet *connected* to which tasks appear. That connection is exactly what Session 7 builds.

15. What Is NOT Here Yet

- No `.map()` — every list is still written out by hand.
- No arrow functions — every handler is a normal, named function.
- No `type` aliases or string-literal unions.
- No `useState<Task[]>` or any collection state — `showTasks` / `searchText` / `currentFilter` are all single, scalar values.
- No callback props.
- No spread syntax.
- No `useEffect` or data fetching.
- No object/props destructuring — props are still read as `props.x` everywhere.
- No Fragment — not taught yet.

If it isn't named above or in Section 14, you haven't missed it — it isn't part of Session 6.

16. Common Mistakes

1. Calling the handler instead of passing it

WRONG:

```
<button onClick={handleShowAll()}>All</button>
```

RIGHT:

```
<button onClick={handleShowAll}>All</button>
```

Why it's wrong: the parentheses call the function immediately, during render, instead of handing React the function to call later, on an actual click. Because `handleShowAll` calls a state setter, calling it during render can also make React render repeatedly and produce an error (see Section 6 for the full explanation).

2. Forgetting `value=` on a controlled input

WRONG:

```
<input onChange={handleSearchChange} />
```

Why it's wrong: the input becomes uncontrolled — `onChange` still runs and `setSearchText` still updates state correctly, but React is no longer deciding what the box displays; the browser is. The state → displayed-input connection is broken.

3. Un-typed event parameters

WRONG:

```
function handleSearchChange(event) {
```

Why it's wrong: this project's strict TypeScript settings will refuse to compile an untyped parameter. The fix is always `event: ChangeEvent<HTMLInputElement>` .

4. Reaching for an arrow function

Do not switch to an arrow function here. Arrow functions are introduced in Session 07. In Session 06, every event handler is a normal named function:

```
function handleSearchChange(event: ChangeEvent<HTMLInputElement>) {  
    setSearchText(event.target.value);  
}
```

```
onChange={handleSearchChange}
```

Those are current-session concepts and are safe.

5. Direct state mutation

WRONG:

```
currentFilter = "completed";
```

RIGHT:

```
setCurrentFilter("completed");
```

Why it's wrong, at two levels:

1. `const [currentFilter, setCurrentFilter] = useState("all");` makes `currentFilter` a `const` binding. TypeScript rejects `currentFilter = "completed";` outright — this line does not even compile, the same way reassigning any other `const` variable never has.
2. Even setting that aside, calling a state setter tells React that this piece of state changed and schedules an update. Changing the variable directly does not tell React that React state changed — only `setCurrentFilter("completed")` does that.

6. Confusing conditional styling with conditional rendering

Why it's a mistake: `className={"filter-button" + (... ? " active" : "")}` picks a *string* — the button is always on the page. `{searchText !== "" ? <p>...</p> : null}` decides whether an *element* exists at all. They look similar but answer different questions.

7. Reaching for `.map()` already

Why it's a mistake: there's still no array of tasks to loop over — the three `TaskItem` calls are still written by hand. `.map()` genuinely belongs to Session 7.

17. Glossary

state A value a component remembers, and that React watches for changes to — created with `useState`.

useState The hook that gives a component a current value, a setter to change it, and an initial value used only on the very first render.

setter The function `useState` returns alongside the current value — the only correct way to change that piece of state.

re-render React running your component function again after a state setter is called, producing updated UI.

event handler A normal function that runs in response to something happening — a click, a keystroke, and so on.

onClick / onChange Props that tell React which function to call when a click or an input change happens.

ChangeEvent<HTMLInputElement> React's type for a change event that came specifically from an `<input>` element.

event.target.value The current text inside the input element that triggered a change event.

controlled input An input whose displayed value is driven by React state (`value={...}`), kept in sync by an `onChange` handler that updates that same state.

conditional styling Choosing a CSS class (or other string) based on a condition — the element itself is always present.

conditional rendering Choosing whether an element exists in the page at all, typically with a ternary-to- `null`.

boolean state State holding `true / false`, commonly flipped with the `!` operator inside a toggle handler.

18. Review Questions

1. Why doesn't a plain variable work for something that should update the screen?

2. What three things does `const [value, setValue] = useState(initial)` give you?
 3. Why does calling the setter cause a re-render, but assigning to the variable directly doesn't?
 4. What's wrong with `onClick={handleShowAll()}`, exactly?
 5. What does `ChangeEvent<HTMLInputElement>` mean, piece by piece?
 6. What makes an input "controlled"?
 7. If you remove `value=` from a controlled input but leave `onChange`, what actually breaks — and what still works?
 8. What is the real difference between conditional styling and conditional rendering?
 9. Why does `!showTasks` work correctly no matter which direction you're toggling?
 10. Why don't the filter buttons and search box actually change which tasks are shown yet?
-

19. Answers

1. Because a plain variable's changes are invisible to React — nothing tells React to run the component again, so the screen never updates even if the variable's value did change.
2. The current value, a setter function to change it, and — only for the very first render — an initial value.
3. `currentFilter` is a `const` binding, so a direct assignment like `currentFilter = "completed";` doesn't even compile. The setter is the correct way to change this state: calling it tells React that this piece of state changed and schedules an update — a signal a plain assignment could never send in the first place.
4. The parentheses call `handleShowAll` immediately, while the line of JSX is being rendered — not when the button is actually clicked. Because `handleShowAll` calls a state setter, calling it during rendering can also cause React to render repeatedly and produce an error. The fix is to pass the function itself, with no parentheses: `onClick={handleShowAll}`.
5. `ChangeEvent` is React's type for a form-element change event; `<HTMLInputElement>` narrows it to specifically an `<input>`, which is what makes `.value` available on `event.target`.
6. Having both `value={someState}` (state controls what's displayed) and `onChange={someHandler}` (the handler keeps that state updated as the user types) wired together on the same input.
7. The box's *display* stops being controlled by state — it starts showing whatever the browser itself renders by default. `onChange` still fires, and the setter still correctly updates React state; only the visual connection back to the input breaks.
8. Conditional styling picks a string (like a CSS class) while the element itself is always on the page. Conditional rendering decides whether the element exists in the page at all.

9. Because `!` always means "the opposite of the current value" — it never needs to know in advance which state it's currently in; it just flips whatever is there right now.

10. Because connecting them to the actual task list requires deriving a filtered subset of a real array and rendering a variable number of rows with `.map()` — and `.map()` hasn't been taught yet. That's exactly what Session 7 builds.

20. What You Should Be Able to Explain Now

- ☐ I can explain why a plain variable can't update the screen, but `useState` can.
- ☐ I can name all three parts of `useState`'s result: current value, setter, initial value.
- ☐ I can explain why calling the setter causes a re-render.
- ☐ I can explain the difference between `onClick={fn}` and `onClick={fn()}.`
- ☐ I can write a normal, named event handler.
- ☐ I can explain `ChangeEvent<HTMLInputElement>` and `event.target.value` at a beginner level.
- ☐ I can build a controlled input using `value` + `onChange` together.
- ☐ I can explain what happens if `value=` is removed but `onChange` stays.
- ☐ I can write a real conditional-rendering ternary that returns `null`.
- ☐ I can explain the difference between conditional styling and conditional rendering.
- ☐ I can explain why the task list still doesn't respond to filtering or searching.

If every box is honestly checked, you understand Session 6 completely. The one open question left is exactly the one your homework's final reflection asks: what would it actually take to make clicking "Completed" hide the pending tasks? That's Session 7.